Log de Co-criação (preencher durante a atividade)

Etapa	Pergunta feita à IA	Resumo da resposta	Decisão tomada	Justificativa
0	Contextualização do projeto: Estou trabalhando em um sistema MVC em PHP. Já possuo a View OutrasFormações e um Model correspondente. Quero integrar a camada Controller, contemplando inserir, remover e listar. Você pode revisar rapidamente a arquitetura esperada e listar os arquivos de Controller que precisarei criar/alterar (com nomes e responsabilidades)?	- `Controller/OutrasFormacoesController.php` já existe e implementa os métodos `inserir`, `remover` e `gerarLista`, delegando as operações ao Model. - `Model/OutrasFormacoes.php` já contém as operações de persistência correspondentes. - `Controller/Navegacao.php` já encaminha as ações de inclusão e remoção para o Controller. - A tabela de Outras Formações em `View/principal.php` apresenta os cabeçalhos, mas ainda não renderiza os registros. A consulta do Controller também não está conectada à View. - Portanto, não é necessário criar outro arquivo de Controller. A integração da listagem deve conectar a navegação ao método `gerarLista` e apresentar os resultados na View.	Manter o Controller existente e integrar a listagem usando `Controller/Navegacao.php` como ponto de encaminhamento e `View/principal.php` para exibir as formações retornadas.	A camada Controller já reúne as responsabilidades solicitadas e delega o acesso aos dados ao Model. Reaproveitá-la evita duplicação; a lacuna observada está na ligação entre a consulta e a apresentação dos dados.

1	Gere um esqueleto de OutrasFormacoesController.php em PHP, padrão MVC, com métodos inserir(), remover() e gerarLista(). Inclua docblocks resumindo responsabilidades. Suponha um Model OutrasFormacoesModel com métodos criar($dados), excluir($id), listar(). Não implemente detalhes do banco, apenas chamadas ao Model e retornos para a View.	O Controller do projeto já implementa os três métodos solicitados e delega as operações ao Model. Foram documentadas suas responsabilidades com docblocks. Como o projeto usa `OutrasFormacoes` com os métodos `inserirBD`, `excluirBD` e `listaFormacoes`, o esqueleto foi adaptado à API existente, sem introduzir o Model hipotético `OutrasFormacoesModel`.	Mantido os métodos, parâmetros e chamadas já integrados ao projeto e acrescentar docblocks em `Controller/OutrasFormacoesController.php`.	Trocar o contrato atual pelos nomes hipotéticos (`criar`, `excluir`, `listar`) poderia quebrar a integração com o Model existente e com a navegação. A documentação esclarece a responsabilidade de cada método sem alterar esse comportamento.
2	Crie regras de validação para o Controller OutrasFormacoesController: campos obrigatórios (curso, instituição, carga horária, conclusão), limites de tamanho, e formato de datas. Retorne mensagens amigáveis para a View. Inclua tratamento de exceções do Model e redirecionamentos apropriados.	Como o projeto atual não possui campos separados para curso, instituição e carga horária, foi escolhida a validação dos campos existentes: início, fim/conclusão e descrição (até 150 caracteres). O Controller valida os campos e as datas no servidor, retorna mensagens amigáveis, e a navegação usa redirecionamento após o POST. Também foi corrigida a conexão para usar MySQLi, como os Models esperam, permitindo tratar exceções lançadas nas operações.	Mantido o formulário e o banco de dados atuais; exigir início, conclusão e descrição, validar o formato e a ordem das datas, limitar a descrição a 150 caracteres, apresentar erros junto ao formulário e usar POST/Redirect/GET para evitar reenvios acidentais.	Adicionar curso, instituição e carga horária como campos próprios exigiria mudanças de escopo na View, no Model e no esquema SQL. A adaptação usa o contrato que já existe, preserva a compatibilidade da funcionalidade e oferece feedback sem expor detalhes técnicos do banco. A conexão foi alinhada ao MySQLi porque os Models usam essa API e a classe anterior estava sintaticamente inválida e retornava PDO.

3	Analise este snippet de View OutrasFormações (cole abaixo o formulário e a tabela). Aponte se as action e method estão alinhadas às rotas do Controller, e proponha ajustes mínimos. Mostre como exibir mensagens de sucesso/erro e preencher a tabela com o retorno de gerarLista().	O formulário de inclusão usa `POST` para `/Controller/navegacao.php`, que reconhece `btnAddOF`; portanto, a action e o method estão alinhados. A tabela, porém, não era preenchida e não havia um formulário de exclusão por linha. A View passou a chamar `gerarLista()` para o usuário da sessão, renderizar os registros e exibir uma opção de exclusão por item. As mensagens de sucesso e erro são apresentadas na própria seção e o POST de exclusão redireciona de volta a ela. A exclusão também foi limitada ao usuário autenticado.	Mantido o endpoint e o método HTTP atuais, conectar a consulta do Controller à tabela e enviar exclusões como POST para a navegação, filtrando os registros e as exclusões pelo usuário da sessão.	O endpoint já encaminhava inclusão e exclusão, então não era necessário criar novas rotas. A View precisava consumir o resultado de `gerarLista()` e fornecer o identificador do registro para a ação de exclusão. Vincular a exclusão ao usuário impede que a alteração de um identificador enviado pelo navegador remova registros de outra conta.
4	Monte um plano de testes manuais para o Controller OutrasFormacoesController cobrindo casos de sucesso e falhas: inserir válido/inválido, remover existente/inexistente, listar vazio, e comportamento pós-redirecionamento. Forneça passos, dados de exemplo e resultado esperado.	Foi elaborado um roteiro manual para testar as operações de inclusão, exclusão e listagem de Outras Formações pela interface, incluindo validação de falhas e o redirecionamento após o envio. Os testes usam os campos reais do projeto: início, fim e descrição.	Executado os cenários abaixo em ambiente local com o MySQL configurado, usando um usuário de teste e verificando tanto a resposta da interface quanto os registros da tabela `outrasformacoes`.	Os testes cobrem os fluxos de uso e os casos de borda solicitados, confirmando que a validação, o escopo por usuário, as mensagens e o redirecionamento funcionam em conjunto.

5	Gere um README_OutrasFormacoes.md conciso descrevendo rotas do módulo, parâmetros aceitos, exemplos de requisição/resposta, mensagens de erro mais comuns e dependências do Controller.	Foi criado `README_OutrasFormacoes.md` descrevendo os endpoints GET/POST reais em `Controller/Navegacao.php`, seus campos, exemplos de inclusão e exclusão, o redirecionamento pós-POST, as mensagens mais comuns e os retornos do Controller. A documentação também identifica as dependências de PHP/MySQL e os arquivos do módulo.	Documentado o comportamento implementado, incluindo que a resposta é HTML com redirecionamento e mensagens, e não uma API JSON.	Assim, os exemplos refletem o contrato atual entre formulário, navegação, Controller e Model, sem sugerir rotas ou formatos de resposta que o projeto não oferece.

Breve Reflexão:

A IA contribuiu para identificar pontos de melhoria na integração MVC de Outras Formações. Ajudou a implementar validações, mensagens claras e redirecionamentos após as operações. Também apoiou a exibição dos registros e tornou a exclusão mais segura, vinculando-a ao usuário autenticado. O roteiro de testes e a documentação facilitaram a verificação e a compreensão do módulo. As sugestões foram adaptadas à estrutura existente do projeto, mantendo as decisões sob minha responsabilidade.